Use Cases

for

Vibecheck

Version 2.0 approved

Prepared by <Joshua, Jun Hong & Marcus>

<GRP 2/SCSA/NTU>

<8/9/2026>

Revision History

Name	Date	Reason For Changes	Version
Joshua	26/10/26	Edited and Formalized Properly Use cases	2

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Guidance for Use Case Template

Document each use case using the template shown in the Appendix. This section provides a description of each section in the use case template.

1. Use Case Identification

1.1. Use Case ID

Give each use case a unique numeric identifier, in hierarchical form: X.Y. Related use cases can be grouped in the hierarchy. Functional requirements can be traced back to a labeled use case.

1.2. Use Case Name

State a concise, results-oriented name for the use case. These reflect the tasks the user needs to be able to accomplish using the system. Include an action verb and a noun. Some examples:

- View part number information.
- Manually mark hypertext source and establish link to target.
- Place an order for a CD with the updated software version.

1.3. Use Case History

1.3.1 Created By

Supply the name of the person who initially documented this use case.

1.3.2 Date Created

Enter the date on which the use case was initially documented.

1.3.3 Last Updated By

Supply the name of the person who performed the most recent update to the use case description.

1.3.4 Date Last Updated

Enter the date on which the use case was most recently updated.

2. Use Case Definition

2.1. Actor

An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks. Different actors often correspond to different user classes, or roles, identified from the customer community that will use the product. Name the actor(s) that will be performing this use case.

2.2. Description

Provide a brief description of the reason for and outcome of this use case, or a high-level description of the sequence of actions and the outcome of executing the use case.

2.3. Preconditions

List any activities that must take place, or any conditions that must be true, before the use case can be started. Number each precondition. Examples:

1. User's identity has been authenticated.
2. User's computer has sufficient free memory available to launch task.

2.4. Postconditions

Describe the state of the system at the conclusion of the use case execution. Number each postcondition. Examples:

1. Document contains only valid SGML tags.
2. Price of item in database has been updated with new value.

2.5. Priority

Indicate the relative priority of implementing the functionality required to allow this use case to be executed. The priority scheme used must be the same as that used in the software requirements specification.

2.6. Frequency of Use

Estimate the number of times this use case will be performed by the actors per some appropriate unit of time.

2.7. Flow of Events

Provide a detailed description of the user actions and system responses that will take place during execution of the use case under normal, expected conditions. This dialog sequence will ultimately lead to accomplishing the goal stated in the use case name and description. This description may be written as an answer to the hypothetical question, "How do I <accomplish the task stated in the use case name>?" This is best done as a numbered list of actions performed by the actor, alternating with responses provided by the system.

2.8. Alternative Flows

Document other, legitimate usage scenarios that can take place within this use case separately in this section. State the alternative course, and describe any differences in the sequence of steps that take place. Number each alternative course using the Use Case ID as a prefix, followed by "AC" to indicate "Alternative Course". Example: X.Y.AC.1.

2.9. Exceptions

Describe any anticipated error conditions that could occur during execution of the use case, and define how the system is to respond to those conditions. Also, describe how the system is to respond if the use

case execution fails for some unanticipated reason. Number each exception using the Use Case ID as a prefix, followed by “EX” to indicate “Exception”. Example: X.Y.EX.1.

2.10. Includes

List any other use cases that are included (“called”) by this use case. Common functionality that appears in multiple use cases can be split out into a separate use case that is included by the ones that need that common functionality.

2.11. Special Requirements

Identify any additional requirements, such as nonfunctional requirements, for the use case that may need to be addressed during design or implementation. These may include performance requirements or other quality attributes.

2.12. Assumptions

List any assumptions that were made in the analysis that led to accepting this use case into the product description and writing the use case description.

2.13. Notes and Issues

List any additional comments about this use case or any remaining open issues or TBDs (To Be Determineds) that must be resolved. Identify who will resolve each issue, the due date, and what the resolution ultimately is.

Use Case For Register Account

Use Case ID:	UC0.1		
Use Case Name:	Register account		
Created By:	Jun Hong	Last Updated By:	Jun Hong
Date Created:	15/10/2026	Date Last Updated:	15/10/2026

Actor:	User (Primary), Firebase Authentication (Supporting)
Description:	The User registers an account so that their assessment results and account information can be associated with the user.
Preconditions:	1. User can open the registration page 2. User has an email address and password
Postconditions:	1. A Firebase User ID is created and the user is signed in 2. Profile names are saved when the profile write succeeds 3. The assessment questionnaire is displayed
Priority:	High
Frequency of Use:	Once per Account
Flow of Events:	1. The system requests first name, last name, email, password and agreement to terms. 2. The User enters details and submits. 3. The System validates required fields and the password length supported by the current form. 4. Firebase creates the identity and returns the authenticated user. 5. The System writes names to the User's profile and navigates to the assessment, or a permitted requested destination.
Alternative Flows:	**UC0.1.AC.1: Use an existing account** 1. The User selects Log in instead of registering. 2. The system opens the login page for UC0.2.
Exceptions:	**UC0.1.EX.1: Invalid registration details** 1. The system identifies missing or rejected details and keeps the form open. 2. The User corrects the details and resubmits. **UC0.1.EX.2: Authentication service unavailable** 1. The system displays a failure and permits a retry. 2. No successful account creation is claimed without acknowledgement. **UC0.1.EX.3: Identity created but name write fails** 1. The current system continues registration but does not claim that names were saved. 2. The User can recover the names through UC5.0; a visible recovery notice is planned.

Includes:	None
Special Requirements:	1. Passwords must not be stored in application data or logs. 2. Required fields and accepted password length must be validated.
Assumptions:	1. The email/password provider is enabled for the application. 2. The User has accepted the required terms before registration
Notes and Issues:	TBDs

Use Case For Authenticate

Use Case ID:	UC0.2		
Use Case Name:	Authenticate, resume and end a session		
Created By:	Jun Hong	Last Updated By:	Jun Hong
Date Created:	15/10/26	Date Last Updated:	15/10/26

Actor:	User, Firebase Authentication
Description:	The User signs in, resumes a session and can end the session when finished.
Preconditions:	1. The User has an existing account for login. 2. An authenticated session exists when the User chooses to log out.
Postconditions:	1. Successful login establishes the selected session persistence. 2. A permitted session is opened 3. Successful logout ends the session and removes access to protected pages
Priority:	High
Frequency of Use:	Every session
Flow of Events:	1. The User enters email/password and chooses Remember me. 2. The System selects local or session persistence and asks Firebase to authenticate. 3. On login success, The system navigates to a permitted destination and supplies the current user to protected pages. 4. The system observes session changes while the application runs.
Alternative Flows:	**UC0.2.AC.1: Remember the session** 1. The User selects Remember me before submitting. 2. The system uses local persistence instead of session-only persistence. **UC0.2.AC.2: Log out** 1. The authenticated User selects Log out. 2. The system signs out and returns to login. **UC0.2.AC.3: Open a protected page without a session** 1. The system redirects the User to login. 2. After successful login, an allowlisted requested session is restored, otherwise the dashboard opens.
Exceptions:	**UC0.2.EX.1: Invalid credentials or throttling** 1. The system displays a login error and grants no new access. 2. The User can correct credentials or retry after the indicated wait.

	UC0.2.EX.2: Network or session verification fails 1. The system displays an error or a reload/retry action. 2. Protected page remains unavailable until the session can be verified.
Includes:	None
Special Requirements:	1. Redirect destinations must be allowlisted application paths. 2. Data access must still be enforced by database rules and server token checks. 3. Password storage and authentication are delegated to Firebase Authentication.
Assumptions:	1. Firebase authentication service can validate the account credentials. 2. Firebase service is not experience any outage
Notes and Issues:	TBDs

Use Case For Create Gamer Profile

Use Case ID:	UC1.0		
Use Case Name:	Create , Take Gamer Personality Assessment		
Created By:	Marcus	Last Updated By:	Marcus
Date Created:	8/9/2026	Date Last Updated:	8/9/2026

Actor:	User
Description:	The Gamer completes or retakes a 15-encounter assessment. The system generates Big Five traits and a gamer archetype, and displays the result.
Preconditions:	1. The user is signed in 2. The assessment questions and scoring algorithm is available
Postconditions:	1. All 15 submitted answers have been validated and scored. 2. Five trait scores and a gamer archetype are displayed. 3. The latest completed result is saved when Firebase Database acknowledges the write. 4. An available local completed result is retained if Firebase fails
Priority:	High
Frequency of Use:	Once / User opts to retake
Flow of Events:	1. The system displays the introduction or resumes the owner's saved draft/result. 2. The user chooses an adventurer name/emblem and begins. 3. The system displays each encounter and its three choices. 4. The user chooses a path; the system records the answer and displays its reaction. 5. The user continues until all 15 encounters are complete. 6. The system validates answers, calculates Big Five traits and determines the archetype. 7. The system caches the completed result, writes the latest-result path with a server timestamp and displays saving/saved/error status. 8. The user views the result or returns home to see updated summary statistics.
Alternative Flows:	**UC1.0.AC.1: Revise an answer** 1. The User selects Back or Change choice. 2. The system returns to the applicable encounter and updates the answer. 3. The User continues the assessment. **UC1.0.AC.2: Save and exit** 1. The User selects Save & exit before completion. 2. The system retains available browser progress and returns home.

	UC1.0.AC.3: Retake the assessment 1. The User selects Play again. 2. The system starts a new attempt while retaining the prior completed result until a replacement is completed
Exceptions:	UC1.0.EX.1: Incomplete questionnaire 1. The system displays a message "Incomplete questionnaire" 2. The system jumps to the question with missing response upon acknowledgement UC1.0.EX.2: System unable to process responses 1. The system displays a message "Failed to generate User Profile, please try again" 2. The system restarts the questionnaire UC1.0.EX.3: Unexpected failure 1. System generates a message "Error, please refresh the page"
Includes:	None
Special Requirements:	1. The scoring result must be produced within the 5-second target in the supplied use-case template; this requires measurement. 2. The UI must prevent accidental response loss where storage is available. 3. Personal traits are recalculated only after a completed retake.
Assumptions:	1. The User provides the responses they intend. 2. The scoring rules are consistently applied
Notes and Issues:	TBDs

Use Case for Game Matching

Use Case ID:	UC1.1		
Use Case Name:	Game Matching		
Created By:	Joshua	Last Updated By:	Joshua
Date Created:	8/9/2026	Date Last Updated:	8/9/2026

Actor:	Gamer, RAWG.io API
Description:	The system uses the Gamer's completed personality profile to display personalised and trending games, with filters, game details and optional favorites.
Preconditions:	1. The User must be successfully logged into the platform. 2. The User must have a completed Big 5 personality profile saved in the system.
Postconditions:	1. A personalised list of recommended games is displayed 2. Selected game details can be inspected. 3. Optional favorite/interaction records are stored only where the current storage mechanism permits.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. The User navigates to the “Game Recommendations” of their dashboard for a catalogue of recommended games 2. The System maps the Big 5 profile to corresponding game themes, mechanics and target genre. 3. The System queries RAWG.io API for currently trending or popular games of mapped genre. 4. The System filters and ranks games returned based on the Big 5 personality profile of the User. 5. The system displays a curated list of matched games organized by genre. 6. The System records the User’s action and updates preference weighting in the background 7. The User selects a game. 8. The System displays the game detail dialog. 9. The User closes the dialog and returns to the lists.
Alternative Flows:	**UC1.1.AC.1:** No genre preference set - After step 3, System detects no preferred genre has been selected - System infers the genre from the Gamer’s Big 5 personality type and play history - System prompts the Gamer to confirm or adjust the inferred genre - Flow resumes at Step 4 **UC1.1.AC.2:** Gamer refines recommendations

	- Gamer applies a filter - System re-queries the RAWG.io API for the updated list based on the filter criteria - System displays the updated recommendation lists - Flow resumes at Step 6 **UC1.1.AC.3: Save a favorite** 1. The Gamer selects the favorite action for a game. 2. The system toggles its locally saved state. **UC1.1.AC.4: No narrow matches** 1. The system broadens to adjacent genres or popularity. 2. The system explains that the results were broadened.
Exceptions:	**UC1.1.EX.1:** RAWG.io API unavailable - The System is unable to retrieve data from the RAWG.IO api - The System displays the most recently cached recommendations and trending list - The System notifies that the list displayed may be outdated **UC1.1.EX.2:** No games match the Gamer's profile - The System broadens the match criteria to adjacent genres or overall popularity rankings, regardless of Big 5 personality. - The System notifies Gamer that no game has been found and that results have been broadened. **UC1.1.EX.3: No completed profile** 1. The system displays the assessment gate. 2. The User can open UC1.0.
Includes:	None
Special Requirements:	System should have the recommended list generated as the Gamer enters their profile
Assumptions:	1. Assumes RAWG.io API returns up to date genre and trending games and that the Gamer has sufficient data for the System to generate a meaningful game match 2. The completed personality profile is valid.
Notes and Issues:	NIL

Use Case Gamer Matching

Use Case ID:	1.2		
Use Case Name:	Recommend Gamer Matching		
Created By:	Joshua	Last Updated By:	Joshua
Date Created:	12/9/26	Date Last Updated:	12/9/26

Actor:	Gamer
Description:	Gamers initiate a matchmaking request, then the matchmaking algorithm evaluates the personality profile of the gamer in conjunction with other Gamers on the platform and recommends them as potential people to match together with according to the Big 5 personality
Preconditions:	- Gamer must be successfully logged into the platform. - Gamer must have a completed Big 5 personality profile saved in the system.
Postconditions:	- A personalized list of other Gamers is displayed - The system logs the matchmaking interaction and session history to refine future algorithmic recommendations.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. Gamer navigates to the <undecided for now> 2. System then matches Gamer with other random Gamers of the same personality type or with personality types that plays well with the Gamer's current personality type 3. System will display a personalized list of Gamers. 4. Gamer can choose to either send a friend request or ignore the System's recommendations 5. System logs down Gamer's action for future recommendations.
Alternative Flows:	1.2 AC.1 Gamer refreshes friend recommendation - System will retry the matching again and displays the new list
Exceptions:	1.2 EX.1 No Gamers of the same personality type - If there are no Gamers of the same personality type, System will just recommend the Gamers whose Big 5 personality type plays well with the current Gamer's personality type. - If no compatible gamers exist at all, System displays an empty state indicating that there is insufficient user base for matching.
Includes:	1. System retrieves the Gamer's Big 5 profile, recorded traits, preferred genre and play history.
Special Requirements:	Friend recommendation list should be generated within 5 seconds
Assumptions:	Gamer has already completed their Big 5 personality profile.

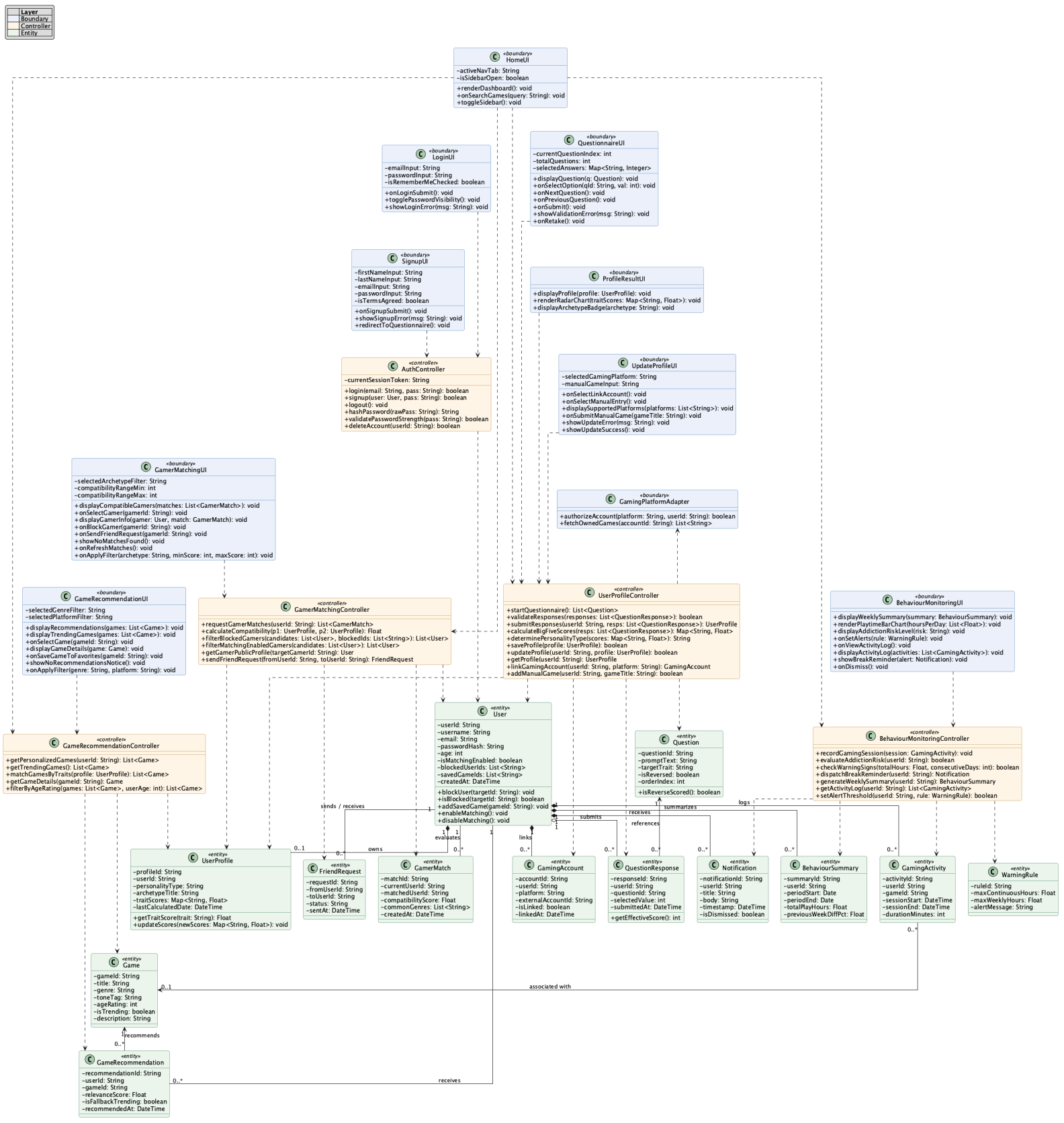
	Assumes there are enough Gamers within the database to random create a new list of Gamers to recommend
Notes and Issues:	NIL

Use Case for Update/Check Profile

Use Case ID:	UC5.0		
Use Case Name:	Update Gamer Profile		
Created By:	Marcus	Last Updated By:	Marcus
Date Created:	17/9/2026	Date Last Updated:	17/9/2026

Actor:	Gamer, Gaming Platform
Description:	The gamer updates their existing Gamer Profile by providing additional gaming data. The Gamer can either link a supported gaming account such as Steam or Epic Games, allowing the system to retrieve their gaming data. The user can also manually enter games they frequently play or enjoy. The system uses the provided information to update the Gamer Profile
Preconditions:	1. Gamer completed the initial Gamer Assessment and an initial Gamer Profile has been generated 2. Gamer has an account and identity has been authenticated 3. The Gamer selected the option to update their Gamer profile
Postconditions:	1. Gamer Profile updated with the additional data and is saved to the account 2. Gaming data retrieved from the linked gaming account has been recorded in Gamer Profile where applicable 3. Manually entered games have been recorded in the Gamer Profile
Priority:	High
Frequency of Use:	Moderate
Flow of Events:	1. Gamer selects the option to Update Gamer Profile 2. System displays options to link a gaming account or manually enter gaming information 3. Gamer selects manually enter gaming information 4. System proceeds to prompt the Gamer to enter additional information 5. System updates Gamer Profile with the additional gaming data 6. System saves the updated Gamer Profile to the Gamer's account 7. System confirms that the Gamer Profile has been successfully updated 8. System gives the option for Gamer to further update their profile or use other features

Alternative Flows:	UC5.0.AC.1: Gamer chooses to link a gaming account instead of manually entering gaming information 1. Gamer selects option to link a gaming account 2. System displays supported gaming platforms 3. Gamer selects the platform 4. System redirects Gamer to the selected gaming platform for authentication and authorisation 5. Gaming platform authenticates the Gamer and grants the System permission to retrieve relevant gaming data 6. System proceeds to Step 5
Exceptions:	UC5.0.EX.1: If the Gamer enters an invalid Gaming Account 1. The System displays a message "Invalid Gaming Account, please try again" 2. System goes back to Step 2 UC5.0.EX.2: If the selected Gaming Platform cannot be reached 1. System displays a message "Data cannot be retrieved, please try again later" 2. System goes back to Step 2 UC5.0.EX.3: If the Gamer enters an invalid manual input 1. System displays a message "Game not found, please re-enter" 2. System goes back to Step 4 UC5.0.EX.4: Unexpected Error 1. System displays a message "User Profile cannot be updated, please try again later" 2. System does not modify the existing Gamer Profile and exits UC5.0
Includes:	None
Special Requirements:	1. System should securely handle authentication information associated with linked gaming accounts 2. System should only retrieve gaming information that the Gamer has authorised the system to access 3. The System should update the Gamer Profile within 5 seconds after valid information is submitted 4. Existing Gamer Profile should not be lost if an update fails
Assumptions:	1. The Gamer has already completed the Gamer Assessment and has an existing Gamer Profile 2. The Gamer is logged into their account 3. Supported Gaming Platforms are able to provide necessary gaming information through an available interface 4. Gamer has the necessary permissions and log in details to access and link their gaming account
Notes and Issues:	1. The specific Gaming Platforms supported by the System are to be finalised


Layer
Boundary
Controller
Entity
«boundary»
HomeUI
-activeNavTab: String
-isSidebarOpen: boolean
+renderDashboard(): void
+onSearchGames(query: String): void
+toggleSidebar(): void
«boundary»
LoginUI
-emailInput: String
-passwordInput: String
-isRememberMeChecked: boolean
+onLoginSubmit(): void
+togglePasswordVisibility(): void
+showLoginError(msg: String): void
«boundary»
QuestionnaireUI
-currentQuestionIndex: int
-totalQuestions: int
-selectedAnswers: Map<String, Integer>
+displayQuestion(q: Question): void
+onSelectOption(qId: String, val: int): void
+onNextQuestion(): void
+onPreviousQuestion(): void
+onSubmit(): void
+showValidationError(msg: String): void
+onRetake(): void
«boundary»
SignupUI
-firstNameInput: String
-lastNameInput: String
-emailInput: String
-passwordInput: String
-isTermsAgreed: boolean
+onSignupSubmit(): void
+showSignupError(msg: String): void
+redirectToQuestionnaire(): void
«boundary»
ProfileResultUI
+displayProfile(profile: UserProfile): void
+renderRadarChart(traitScores: Map<String, Float>): void
+displayArchetypeBadge(archetype: String): void
«controller»
AuthController
-currentSessionToken: String
+login(email: String, pass: String): boolean
+signup(user: User, pass: String): boolean
+logout(): void
+hashPassword(rawPass: String): String
+validatePasswordStrength(pass: String): boolean
+deleteAccount(userId: String): boolean
«boundary»
UpdateProfileUI
-selectedGamingPlatform: String
-manualGameInput: String
+onSelectLinkAccount(): void
+onSelectManualEntry(): void
+displaySupportedPlatforms(platforms: List<String>): void
+onSubmitManualGame(gameTitle: String): void
+showUpdateError(msg: String): void
+showUpdateSuccess(): void
«boundary»
GamerMatchingUI
-selectedArchetypeFilter: String
-compatibilityRangeMin: int
-compatibilityRangeMax: int
+displayCompatibleGamers(matches: List<GamerMatch>): void
+onSelectGamer(gamerId: String): void
+displayGamerInfo(gamer: User, match: GamerMatch): void
+onBlockGamer(gamerId: String): void
+onSendFriendRequest(gamerId: String): void
+showNoMatchesFound(): void
+onRefreshMatches(): void
+onApplyFilter(archetype: String, minScore: int, maxScore: int): void
«boundary»
GamingPlatformAdapter
+authorizeAccount(platform: String, userId: String): boolean
+fetchOwnedGames(accountId: String): List<String>
«boundary»
GameRecommendationUI
-selectedGenreFilter: String
-selectedPlatformFilter: String
+displayRecommendations(games: List<Game>): void
+displayTrendingGames(games: List<Game>): void
+onSelectGame(gameId: String): void
+displayGameDetails(game: Game): void
+onSaveGameToFavorites(gameId: String): void
+showNoRecommendationsNotice(): void
+onApplyFilter(genre: String, platform: String): void
«controller»
GamerMatchingController
+requestGamerMatches(userId: String): List<GamerMatch>
+calculateCompatibility(p1: UserProfile, p2: UserProfile): Float
+filterBlockedGamers(candidates: List<User>, blockedIds: List<String>): List<User>
+filterMatchingEnabledGamers(candidates: List<User>): List<User>
+getGamerPublicProfile(targetGamerId: String): User
+sendFriendRequest(fromUserId: String, toUserId: String): FriendRequest
«controller»
UserProfileController
+startQuestionnaire(): List<Question>
+validateResponses(responses: List<QuestionResponse>): boolean
+submitResponses(userId: String, resps: List<QuestionResponse>): UserProfile
+calculateBigFiveScores(resps: List<QuestionResponse>): Map<String, Float>
+determinePersonalityType(scores: Map<String, Float>): String
+saveProfile(profile: UserProfile): boolean
+updateProfile(userId: String, profile: UserProfile): boolean
+getProfile(userId: String): UserProfile
+linkGamingAccount(userId: String, platform: String): GamingAccount
+addManualGame(userId: String, gameTitle: String): boolean
«boundary»
BehaviourMonitoringUI
+displayWeeklySummary(summary: BehaviourSummary): void
+renderPlaytimeBarChart(hoursPerDay: List<Float>): void
+displayAddictionRiskLevel(risk: String): void
+onSetAlerts(rule: WarningRule): void
+onViewActivityLog(): void
+displayActivityLog(activities: List<GamingActivity>): void
+showBreakReminder(alert: Notification): void
+onDismiss(): void
«controller»
GameRecommendationController
+getPersonalizedGames(userId: String): List<Game>
+getTrendingGames(): List<Game>
+matchGamesByTraits(profile: UserProfile): List<Game>
+getGameDetails(gameId: String): Game
+filterByAgeRating(games: List<Game>, userAge: int): List<Game>
«entity»
User
-userId: String
-username: String
-email: String
-passwordHash: String
-age: int
-isMatchingEnabled: boolean
-blockedUserIds: List<String>
-savedGameIds: List<String>
-createdAt: DateTime
+blockUser(targetId: String): void
+isBlocked(targetId: String): boolean
+addSavedGame(gameId: String): void
+enableMatching(): void
+disableMatching(): void
«entity»
Question
-questionId: String
-promptText: String
-targetTrait: String
-isReversed: boolean
-orderIndex: int
+isReverseScored(): boolean
«controller»
BehaviourMonitoringController
+recordGamingSession(session: GamingActivity): void
+evaluateAddictionRisk(userId: String): boolean
+checkWarningSigns(totalHours: Float, consecutiveDays: int): boolean
+dispatchBreakReminder(userId: String): Notification
+generateWeeklySummary(userId: String): BehaviourSummary
+getActivityLog(userId: String): List<GamingActivity>
+setAlertThreshold(userId: String, rule: WarningRule): boolean
«entity»
UserProfile
-profileId: String
-userId: String
-personalityType: String
-archetypeTitle: String
-traitScores: Map<String, Float>
-lastCalculatedDate: DateTime
+getTraitScore(trait: String): Float
+updateScores(newScores: Map<String, Float>): void
«entity»
FriendRequest
-requestId: String
-fromUserId: String
-toUserId: String
-status: String
-sentAt: DateTime
«entity»
GamerMatch
-matchId: String
-currentUserId: String
-matchedUserId: String
-compatibilityScore: Float
-commonGenres: List<String>
-createdAt: DateTime
«entity»
GamingAccount
-accountId: String
-userId: String
-platform: String
-externalAccountId: String
-isLinked: boolean
-linkedAt: DateTime
«entity»
QuestionResponse
-responseId: String
-userId: String
-questionId: String
-selectedValue: int
-submittedAt: DateTime
+getEffectiveScore(): int
«entity»
Notification
-notificationId: String
-userId: String
-title: String
-body: String
-timestamp: DateTime
-isDismissed: boolean
«entity»
BehaviourSummary
-summaryId: String
-userId: String
-periodStart: Date
-periodEnd: Date
-totalPlayHours: Float
-previousWeekDiffPct: Float
«entity»
GamingActivity
-activityId: String
-userId: String
-gameId: String
-sessionStart: DateTime
-sessionEnd: DateTime
-durationMinutes: int
«entity»
WarningRule
-ruleId: String
-maxContinuousHours: Float
-maxWeeklyHours: Float
-alertMessage: String
«entity»
Game
-gameId: String
-title: String
-genre: String
-toneTag: String
-ageRating: int
-isTrending: boolean
-description: String
«entity»
GameRecommendation
-recommendationId: String
-userId: String
-gameId: String
-relevanceScore: Float
-isFallbackTrending: boolean
-recommendedAt: DateTime
sends / receives
1
1
evaluates
1
1
owns
0..1
0..*
links
1
1
0..*
0..*
submits
1
1
receives
references
1
1
summarizes
1
1
logs
1
1
0..*
0..*
0..*
0..*
0..*
0..*
0..*
0..*
0..*
associated with
0..1
0..*
recommends
0..*
1
receives
0..*

vibecheck.gg Quiz Archetypes Research

The Strategist

Filter

Genres

Platform

Saved Games

[span_3]

CURATED STRATEGIST



Factorio

Automation & Simolation

Algorithmic thinking

Save to Favorites



Civilization VI

4X Grand Strategy

Recursive planning

View Details



XCOM 2

Turn Based Tactics

Calculated Risk Mitigation

View Details

TRENDING GAMES



Factorio

View Details



Civilization VI

View Details



XCOM 2

View Details



Rronrant

View Details



XCOM

View Details

vibecheck.gg Quiz Archetypes Research

BEHAVIOUR MONITORING DASHBOARD

Set Alerts | Activity Log

LAST 7 DAYS	MAX SESSION	CURRENT SESSION
28.5h	5.2h	32min
Trend: ↓12%	Civilization VI	Factorio



Notification Center

- ⚠ Warning: Factorio session exceeded 5 hours ×
- ⚠ Warning: Factorio session exceeded 5 hours ×

Addiction Risk Level

Low — High

Addiction Risk Level

⚠ Low (addictien state when high
pscel with caution state when
high

Notification Center

- ⚠ Warning: Factorio session exceeded 5 hours (Dismiss, Set Timer) ×
- ⚠ Warning: Factorio session exceeded 5 hours (Dismiss, Set Timer) ×

Activity Log

vibecheck.gg Quiz Archetypes Research

POTENTIAL MATCHES

[span_2]



NeonSpecter
Major Archetypes
Minor Archetypes



MATCH SCORE:
94%

VIEW PROFILE BLOCK

MOMPATIBILTY 2]



GladiatorPro
Major Archetypes
Minor Archetypes



MATCH SCORE:
94%

VIEW PROFILE BLOCK

MOMPATIBIUTY 2]



CyberExplorer
Major Archetypes
Minor Archetypes



MATCH SCORE:
94%

FILTER

Filter by Archetype

Archetype

Compatibility

None

Compatibility Range

0 900

VIEW PROFILE BLOCK

[span_3]

Use Case Diagram

